
Métricas de Avaliação

Aula 5,5 — Precisão, Recall, o “@ k ”, MAP, nDCG e MRR

Projeto Integrador III ■ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

Aula 05 — feito

- Necessidade $\times$ consulta
- Escala 0/1/2
- *Pooling*
- κ de Cohen

Temos um **gabarito**.

Hoje

- Precisão, Recall, o “@ k ”
- MAP
- MRR
- nDCG

O que se **calcula** com ele.

Toda conta de hoje é verificável na calculadora. Se não bater, alguém errou — e queremos saber quem.

Necessidade de informação (Aula 05)

“Quais são os modelos formais que um motor de busca usa para ordenar documentos?”

Consulta: modelo de recuperacao

ranking do BM25	d3	d1	d2	d4	d8	d6	d5	d7
grau julgado	2	1	2	0	0	1	0	0

Gabarito completo: d2 e d3 com grau **2**; d1 e d6 com grau **1**; o resto **0**.

Lembrete do slide que fecha a Aula 05: o BM25 pôs d1 em 2º só porque ele contém “recuperacao”.

```
# --- o que o sistema devolveu (saida do BM25 da Aula 04) -----
ranking <- c("d3","d1","d2","d4","d8","d6","d5","d7")

# --- o gabarito: graus julgados na Aula 05, ANTES de rodar -----
grau <- c(d1 = 1, d2 = 2, d3 = 2, d4 = 0,
          d5 = 0, d6 = 1, d7 = 0, d8 = 0)

# --- versao binaria: "relevante" = grau 2 (limiar escolhido) -----
relevantes <- names(grau)[grau == 2]      # -> "d2" "d3"

# rel[i] = 1 se o documento da posicao i do ranking e relevante
rel <- as.integer(ranking %in% relevantes)
rel

[1] 1 0 1 0 0 0 0 0
```

rel é o **gabarito reordenado pelo ranking**: 1 na posição 1 (d3) e na 3 (d2).

Precisão e Recall: duas perguntas diferentes

$$\text{Precisão} = \frac{\text{relevantes recuperados}}{\text{recuperados}} \qquad \text{Recall} = \frac{\text{relevantes recuperados}}{\text{relevantes no } \textit{corpus}}$$

- **Precisão** — “do que eu *mostrei*, quanto presta?”
O denominador é o que o usuário vê.
- **Recall** — “do que *presta*, quanto eu mostrei?”
O denominador só o **gabarito** conhece.

Sem gabarito, a precisão ainda é estimável olhando os resultados um a um. **O recall não é** — exige saber quantos relevantes existem no *corpus* inteiro.

A tabela de contingência

	relevante	não relevante
recuperado	VP	FP
não recuperado	FN	VN

$$\text{Precisão} = \frac{VP}{VP + FP}$$

$$\text{Recall} = \frac{VP}{VP + FN}$$

VP verdadeiro positivo ▪ **FP** falso positivo (lixo no topo) ▪ **FN** falso negativo (relevante que ficou de fora) ▪ **VN** verdadeiro negativo.

É a mesma tabela da classificação — com um problema: ela supõe um **conjunto** recuperado e **ignora a ordem**. Um motor de busca devolve uma *lista ordenada* de tudo. Daí o “@k”.

O que é o “@ k ”

Leia como “nos k primeiros”

P@3 = precisão considerando **só os 3 primeiros** resultados.

R@10 = recall considerando só os 10 primeiros.

Por que existe. O sistema não separa em “recuperado” e “não recuperado”: ele **ordena tudo**. Onde termina “recuperado”? O “@ k ” responde: **corte a lista na posição k** , chame de recuperado o que ficou acima do corte, e calcule a métrica ali.

k não é arbitrário — escolha o k que o *seu* usuário realmente olha.

Busca web: $k = 10$ (uma página). Celular: $k = 3$. Sistema de recomendação: $k = 5$.

Calculando $P@k$ e $R@k$ para todo k

```
R <- length(relevantes)  # 2 relevantes existem no corpus
k <- seq_along(ranking)  # posicoes de corte: 1, 2, ..., 8

# cumsum: quantos relevantes ja apareceram ate a posicao k
acertos <- cumsum(rel)

precisao <- acertos / k  # acertos sobre o que foi MOSTRADO
recall   <- acertos / R  # acertos sobre o TOTAL de relevantes

tabela <- round(rbind(P_at_k = precisao, R_at_k = recall), 3)
colnames(tabela) <- paste0("k=", k)
tabela
```

	k=1	k=2	k=3	k=4	k=5	k=6	k=7	k=8
P_at_k	1.0	0.5	0.667	0.5	0.4	0.333	0.286	0.25
R_at_k	0.5	0.5	1.000	1.0	1.0	1.000	1.000	1.00

O cumsum é o truque: transforma o vetor rel de 0/1 na contagem acumulada de acertos.

Lendo a tabela: o dente de serra

k	1	2	3	4	5	6	7	8
doc	d3	d1	d2	d4	d8	d6	d5	d7
rel	1	0	1	0	0	0	0	0
$P@k$	1,00	0,50	0,67	0,50	0,40	0,33	0,29	0,25
$R@k$	0,50	0,50	1,00	1,00	1,00	1,00	1,00	1,00

- A **precisão** sobe quando cai um relevante e desce a cada não relevante — o “dente de serra”.
- O **recall** só sobe, em degraus, e trava em 1 quando o último relevante aparece.
- Depois de $k = 3$ o recall não aprende mais nada; a precisão só piora.

O compromisso entre os dois

Aumentar k

Recall **sobe** (ou fica igual).

Precisão tende a **cair**.

Diminuir k

Precisão **sobe**.

Recall **cai**.

Nos extremos:

- Devolver o *corpus* inteiro $\Rightarrow$ recall = 1, precisão péssima.
- Devolver 1 documento certo $\Rightarrow$ precisão = 1, recall minúsculo.

Por isso **nenhuma das duas sozinha** avalia um motor de busca, e por isso as três métricas seguintes existem: todas tentam resumir a **lista inteira** em um número que leve a **ordem** em conta.

Average Precision: por que $P@k$ não basta

Problema. $P@3$ vale 0,667 nos dois rankings abaixo:

ranking A	R	R	N
ranking B	N	R	R

Dois relevantes em três posições, nos dois casos. Mas **A é claramente melhor**: o usuário acha o que quer na primeira linha.

A ideia do AP

Calcular a precisão **só nas posições onde caiu um relevante** e tirar a média. Relevante que aparece **cedo** entra com precisão alta; **tarde**, com precisão baixa.

$$AP = \frac{1}{R} \sum_{k: rel_k=1} P@k$$

AP no nosso ranking, passo a passo

posição	doc	rel	P@k	entra na soma?
1	d3	1	$1/1 = 1,000$	sim
2	d1	0	$1/2 = 0,500$	não
3	d2	1	$2/3 = 0,667$	sim
4	d4	0	$2/4 = 0,500$	não
... nenhum relevante daí em diante				

$$AP = \frac{1,000 + 0,667}{2} = \mathbf{0,833}$$

O divisor é $R = 2$: o número de relevantes **no corpus**, não o número de parcelas da soma.

```
# as precisoes apenas nas posicoes em que caiu um relevante
precisao[rel == 1]

[1] 1.0000000 0.6666667

# soma dessas precisoes, dividida por R (nao pelo n. de parcelas)
ap <- sum(precisao[rel == 1]) / R
round(ap, 3)

[1] 0.833
```

`rel == 1` produz um vetor de TRUE/FALSE; usá-lo como índice **filtra** o vetor `precisao`. É o mesmo idioma de `x[x > 0]`.

A pegadinha do divisor

Por que $\frac{1}{R}$, e não a média simples das parcelas?

Um experimento mental

10 relevantes no *corpus*. Um sistema recupera **um só**, na posição 1.

- Média simples das parcelas: 1,000 — **nota máxima**. Errado: ele perdeu 9.
- Dividindo por $R = 10$: 0,100. **Certo**.

Cada relevante que **não** foi recuperado entra na soma valendo 0.

Consequência: o **AP embute o recall**. Não é uma métrica só de precisão, apesar do nome.

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

- AP é de **uma** consulta; MAP é a média sobre **todas** as consultas Q .
- Com uma consulta só, $\text{MAP} = \text{AP}$ — e não significa quase nada.
- O TREC usa tipicamente **50** tópicos. Não é capricho: com poucas consultas a variância domina o resultado.

Ordem de grandeza

O AP de *uma* consulta pode variar 0,3 entre dois sistemas por puro acaso. Sobre 50 consultas, uma diferença de 0,02 no MAP já pode ser real.

MRR: quando basta *uma* resposta

$$RR = \frac{1}{\text{posição do primeiro relevante}} \quad MRR = \frac{1}{|Q|} \sum_{q \in Q} RR(q)$$

posição 1 $\rightarrow$ 1,00 2 $\rightarrow$ 0,50 3 $\rightarrow$ 0,33 10 $\rightarrow$ 0,10

```
which(rel == 1)           # posicoes de TODOS os relevantes  
  
[1] 1 3  
  
mrr <- 1 / which(rel == 1)[1]  # [1] pega so a primeira delas  
mrr  
  
[1] 1
```

Ignora todo o resto da lista — por isso só serve quando uma resposta encerra a busca: *question answering*, busca de fórmula, “qual o CEP de...”.

nDCG: o problema que ele resolve

P@k, MAP e MRR tratam relevância como **sim ou não**. Mas no nosso gabarito d2 tem grau 2 e d1 tem grau 1 — e as binárias jogam d1 fora.

Três ideias, escondidas no nome

G (*gain*) — cada documento traz um ganho igual ao seu **grau**.

DC (*discounted cumulative*) — soma os ganhos, **descontando por posição**.

n (*normalized*) — divide pelo **melhor ranking possível**, para ficar em $[0, 1]$.

$$\text{DCG}@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}$$

$$\text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}$$

Por que $\log_2(i + 1)$ e não $1/i$?

Queremos um desconto que caia com a posição, mas **devagar**.

posição i	$\log_2(i + 1)$	desconto $1/\log_2(i + 1)$	desconto $1/i$
1	1,00	1,000	1,000
2	1,58	0,631	0,500
3	2,00	0,500	0,333
5	2,58	0,387	0,200
10	3,46	0,289	0,100

Com $1/i$ a posição 10 vale **10×** menos que a primeira — a cauda da lista deixa de contar. Com o log, vale **3,5×** menos. O “+1” existe para que $\log_2(1 + 1) = 1$ na primeira posição, evitando divisão por zero.

A escolha do logaritmo é **empírica**, não derivada: Järvelin & Kekäläinen (2002) a propuseram por se comportar bem, e ela pegou.

DCG no nosso ranking, com os graus

posição i	doc	grau rel_i	$\log_2(i + 1)$	parcela
1	d3	2	1,000	$2/1,000 = 2,000$
2	d1	1	1,585	$1/1,585 = 0,631$
3	d2	2	2,000	$2/2,000 = 1,000$
4	d4	0	2,322	0
5	d8	0	2,585	0
6	d6	1	2,807	$1/2,807 = 0,356$
7	d5	0	3,000	0
8	d7	0	3,170	0

$$DCG = 2,000 + 0,631 + 1,000 + 0,356 = \mathbf{3,987}$$

O ranking ideal e a normalização

O **IDCG** é o DCG do melhor ranking possível: os *mesmos* graus, ordenados do maior para o menor.

graus obtidos	2	1	2	0	0	1	0	0
graus ideais	2	2	1	1	0	0	0	0

$$\text{IDCG} = \frac{2}{1,000} + \frac{2}{1,585} + \frac{1}{2,000} + \frac{1}{2,322} = 4,193$$

$$\text{nDCG} = \frac{3,987}{4,193} = \mathbf{0,951}$$

Em R o ranking ideal é literalmente `sort(grau, decreasing = TRUE)` — por isso a fórmula do IDCG some no código: ela vira um `sort`.

```
# desconta cada ganho pela sua posicao e soma tudo
dcg <- function(r) sum(r / log2(seq_along(r) + 1))

# o ideal e o MESMO vetor de ganhos, ordenado do maior ao menor
ndcg <- dcg(rel) / dcg(sort(rel, decreasing = TRUE))
round(ndcg, 3)

[1] 0.92
```

`seq_along(r)` dá $1, 2, \dots, n$; `log2(seq_along(r) + 1)` dá o vetor de descontos. Tudo é vetorizado — não há laço.

nDCG com relevância graduada — o uso de verdade

```
# grau de cada documento, NA ORDEM em que o sistema os devolveu
g <- grau[ranking]
g

d3 d1 d2 d4 d8 d6 d5 d7
  2  1  2  0  0  1  0  0

ndcg_g <- dcg(g) / dcg(sort(g, decreasing = TRUE))
round(ndcg_g, 3)

[1] 0.951
```

`grau[ranking]` reordena o gabarito pelo ranking — indexar um vetor nomeado por um vetor de nomes.

Com graus, d1 e d6 passam a contar; a nota sobe de 0,920 para 0,951.

```
resumo <- c(P_at_3 = precisao[[3]], AP = ap, MRR = mrr,  
            nDCG_bin = ndcg, nDCG_grad = ndcg_g)  
round(resumo, 3)
```

P_at_3	AP	MRR	nDCG_bin	nDCG_grad
0.667	0.833	1.000	0.920	0.951

Note o `[[3]]` em `precisao[[3]]`: com colchete simples o valor viria com o nome da posição grudado, como na Aula 04.

Cinco números para **um** ranking. Nenhum deles é “a” nota do sistema.

Qual métrica usar?

- **P@ k** — quando o usuário olha só as primeiras k respostas. Fácil de explicar a quem não é da área.
- **MAP** — visão global da qualidade do *ranking*, com relevância binária. É o padrão para comparar sistemas.
- **nDCG** — quando há níveis de relevância (bom vs. ótimo). Padrão em busca web e recomendação.
- **MRR** — quando basta *uma* resposta certa: QA, busca de fórmula, RAG.

Regra prática

Relatar sempre **mais de uma** métrica, e sempre dizer o k . “MAP = 0,83” sozinho não permite a ninguém julgar seu sistema.

- ① **Uma consulta só.** Tudo o que calculamos hoje veio de *uma* consulta. Serve para entender a mecânica, não para concluir nada. Precisamos de dezenas.
- ② **Corpus pequeno.** Com 8 documentos todas as métricas ficam altas e próximas (0,83 a 1,00) — elas **não discriminam**. É um sintoma, não um resultado bom.
- ③ **Diferença pequena não é diferença.** MAP 0,84 contra 0,83 não significa que um sistema é melhor. Compara-se com **teste estatístico** sobre os APs consulta a consulta (Aula 16).

As três são formas do mesmo erro: tratar um número como se não tivesse variância.

Tarefa 1 — explicar e explorar o código

Para **cada** *chunk* da aula: explicar linha a linha o que ele faz e por quê; depois, **alterar** algo e reportar o efeito. Sugestões:

- mudar o limiar binário para `grau >= 1` e recalcular tudo;
- inverter o ranking (`rev(ranking)`) e ver cada métrica desabar — qual cai mais? por quê?
- trocar `log2(i+1)` por `i` no `dcg` e comparar o `nDCG`;
- mover `d6` da 6ª para a 4ª posição: o que muda em cada métrica, e o que **não** muda?

Tarefa 2 — avaliar de verdade

Com as 3 consultas e o gabarito que vocês julgaram na Aula 05, avaliar **TF-IDF** (Aula 02) e **BM25** (Aula 04) com `P@3`, `MAP` e `nDCG`. Dizer qual venceu, e por quê — olhando também as consultas *individuais*, não só a média.

Próxima aula — 06

- Realimentação de relevância
- Rocchio e *pseudo-feedback*
- O gabarito deixa de ser régua e vira **entrada do algoritmo**

Leitura

- Manning et al., *IIR* — cap. 8
- Baeza-Yates & Ribeiro-Neto — cap. 4
- Järvelin & Kekäläinen (2002), o artigo do nDCG

A ideia para levar

Toda métrica escolhe **o que conta como sucesso**.

Escolher a métrica errada é otimizar o sistema para a coisa errada — e o sistema vai obedecer.

Obrigado!

Dúvidas? Vamos ao R.
